

AgentIA — The Spec-Driven Design

Every specification the platform reads, and where each one is consumed

AgentIA Engineering

2026-09-29

What this document is. The platform is spec-first: it ingests formal specifications, generates against them, and verifies against them. This document catalogues every specification in force — the blueprint, the Spec-Kit document, the constitution, the per-stage instruction documents, the dependency allowlist, the contract suite, and the skill format — and states where each is read. The design claim is that the specification, not the model, is the authority.

Contents

1	The spine	1
2	The blueprint (the primary spec)	1
3	The Spec-Kit document (the human-facing spec)	2
4	The constitution (the governance spec)	2
5	The instruction documents (the procedural spec)	2
6	The dependency allowlist (the build spec)	2
7	The contract suite (the behavioural spec)	3
8	The skill format (the optimisation spec)	3
9	Where each spec is consumed	3

1 The spine

Generation is a function of a specification: *spec* → *generate* → *verify against the spec*. The model proposes; the specification constrains. The specifications below are the parts of that function, in order of how directly they determine the output.

2 The blueprint (the primary spec)

The `ArchitectureBlueprint` (`models/blueprint.py`) is the machine-checkable specification every generation session starts from. Its schema:

Field	Meaning
<code>serviceName</code>	lowercase-hyphen service identity
<code>packageName</code>	base Java package
<code>basePort</code>	the port the service binds (1024–65535)
<code>databaseMode</code>	datasource dialect

<code>layers[]</code>	the prescribed layer list (controller/service/repository/model)
<code>entities[]</code>	the domain entities
<code>userStories[]</code>	the acceptance stories (Given/When/Then)

Each entity carries `name`, `tableName`, and `attributes` (name, type, nullable, isPrimaryKey, validation-Rules); each story carries `id`, `priority`, `role`, `intent`, `benefit`, and `scenarios` (scenarioId, given, when, then). `validate_blueprint` enforces structural and constitutional integrity before a session may start.

3 The Spec-Kit document (the human-facing spec)

A human writes a `spec.md` conforming to Spec Kit; `spec_service.parse_spec_markdown` parses it into an `ArchitectureBlueprint`, `validate_blueprint` checks it, and `save_specification` stores it under a `specId`. This is the button visible in the UI as “*Fase 4: Ingesta y Validación Formal de Blueprints (Spec Kit)*”, and the export button in the Requirements tab (“*Descargar documento spec.md compatible con Spec Kit*”).

4 The constitution (the governance spec)

`.specify/memory/constitution.md` (v1.1.0) is the six-principle authority the generator and the compliance gate both defer to:

1. **Strict layering** — controller → service → repository → model, each layer talking only to the one below it.
2. **Immutable contracts, early validation** — records only, no JPA entities at the boundary, Jakarta Validation on requests.
3. **Centralised errors** — one `@RestControllerAdvice`, one uniform RFC 7807 shape.
4. **Offline determinism** — hermetic `mvn test -o` in a sealed container; no undeclared dependencies, no build-time downloads.
5. **Quality gates and bounded repair** — 100% test pass; at most three auto-repair iterations, then BLOCKED.
6. **Secrets and the LangGraph boundary** — no hardcoded keys; the LLM orchestrator stays outside the generated service; no LLM calls from tests.

5 The instruction documents (the procedural spec)

The five stage documents (`resources/instructions/*.md`) are a per-stage procedural specification: each prescribes the technology contract, the rules, the output contract, and the prohibitions for one stage. The manifest maps stage to document; the set is versioned content-addressed, so an artifact records exactly which instruction text produced it. (Full text in the companion report *Agents, Prompts and the LangGraph*.)

6 The dependency allowlist (the build spec)

`resources/dependency_allowlist.json` is the build’s specification: the parent, the five allowed dependencies, and the three build plugins. Anything not listed is a blocking violation at the scaffold stage, because the offline build cannot resolve it.

7 The contract suite (the behavioural spec)

Nineteen contract documents, one per feature (001–015), specify behaviour at the seam where it can be enforced:

Contract	Specifies
011 .../instruction-document.md	the four-section shape the loader requires
011 .../compliance-verdict.md	the verdict and LOCAL/ACCUMULATED attribution
011 .../stage-execution.md	the stage seam and its failure policy
012 .../verification-result.md	the three-state verifier result
012 .../verification-observability.md	how a substituted result is marked
013 .../cost-record.md, cost-report.md	cost record and report shape
014 .../edits.md, gate.md, skill-document.md	the skill edit ops, the gate, the doc format
015 .../diagnostic-record.md, contribution-measurement.md, evolution-round.md	diagnostics, leave-one-out, the round

Together they are the platform’s own specification of itself: each feature ships a `spec.md`, a `data-model.md` and its contracts before its code, and the tests are written against those documents.

8 The skill format (the optimisation spec)

The skill document (`contracts/skill-document.md`) has a fixed structure — title, granularity, “when to apply”, numbered rules, and a protected `SLOW_UPDATE` region. It is the shape a reflective optimiser may edit and the boundary it may not cross.

9 Where each spec is consumed

Specification	Consumed by
Blueprint / <code>spec.md</code>	<code>spec_service</code> (parse, validate, store), every stage’s task payload
Constitution	compliance gate, stage instruction docs, repair cap
Instruction documents	<code>instructions.py</code> loader, <code>render_stage_request</code>
Dependency allowlist	scaffolder compliance check
Contracts	the test suite (written against them), data models
Skill document	<code>skills/document.py</code> , the injection prefix